



Manual de Instalación — PathFinderAI

Guía paso a paso para instalar, configurar y ejecutar **PathFinderAI** en un entorno de desarrollo local.

Proyecto Final de Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW) — I.E.S. «Venancio Blanco» (Salamanca)

Índice

1. [Requisitos previos](#)
2. [Configuración del entorno](#)
3. [Inicialización de la base de datos](#)
4. [Ejecución del sistema](#)
5. [Ejecución de tests](#)
6. [Verificación final](#)
7. [Resolución de problemas](#)

1. Requisitos previos

Antes de instalar PathFinderAI, asegúrese de disponer del siguiente software y cuentas de servicio.

1.1. Software necesario

Herramienta	Versión mínima	Comprobación	Descarga
Node.js	18.x o superior	<code>node --version</code>	https://nodejs.org/
npm	9.x o superior	<code>npm --version</code>	(incluido con Node.js)
Git	2.x o superior	<code>git --version</code>	https://git-scm.com/
Editor de código	—	—	Visual Studio Code (recomendado)
Navegador moderno	—	—	Chrome, Edge o Firefox

Captura 1.1 — Terminal mostrando las versiones de Node.js, npm y Git instaladas:

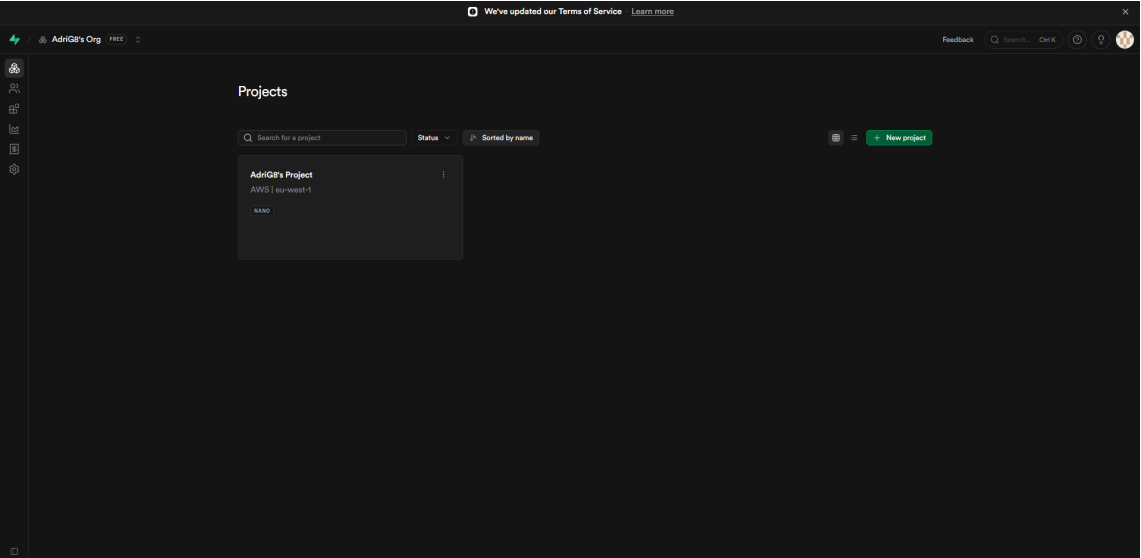
```
PS C:\Users\adrian.gomez\Desktop\PathFinderAI> node --version
v24.13.0
PS C:\Users\adrian.gomez\Desktop\PathFinderAI> npm --version
11.6.2
PS C:\Users\adrian.gomez\Desktop\PathFinderAI> git --version
git version 2.52.0.windows.1
PS C:\Users\adrian.gomez\Desktop\PathFinderAI>
```

1.2. Cuentas de servicios externos

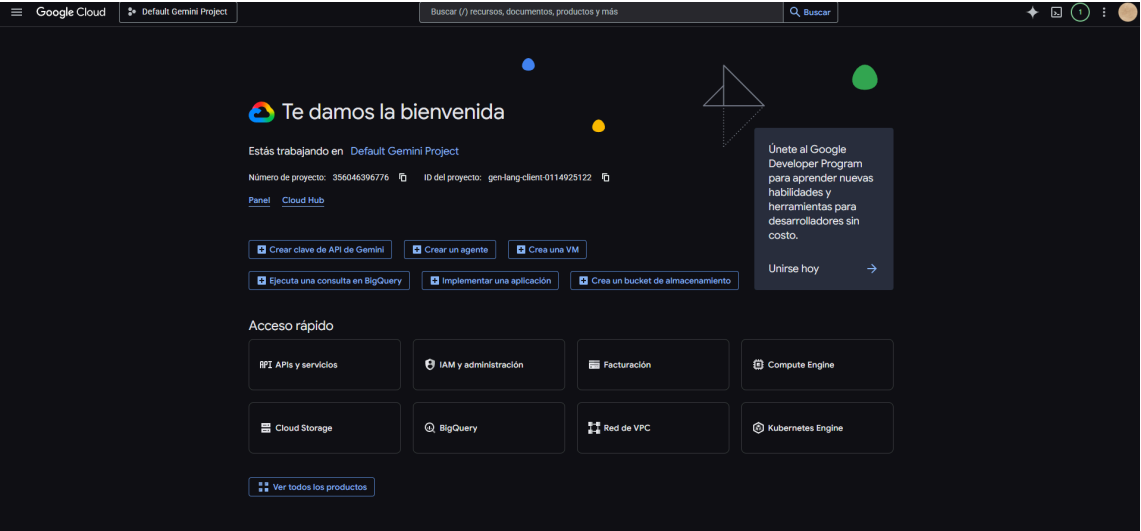
PathFinderAI depende de tres servicios externos. Cree una cuenta gratuita en cada uno:

Servicio	Para qué se usa	Enlace
Supabase	Base de datos PostgreSQL y autenticación	https://supabase.com
Google Cloud	OAuth 2.0 (inicio de sesión con Google)	https://console.cloud.google.com/
Google AI Studio	Clave de API para Gemini	https://makersuite.google.com/app/apikey

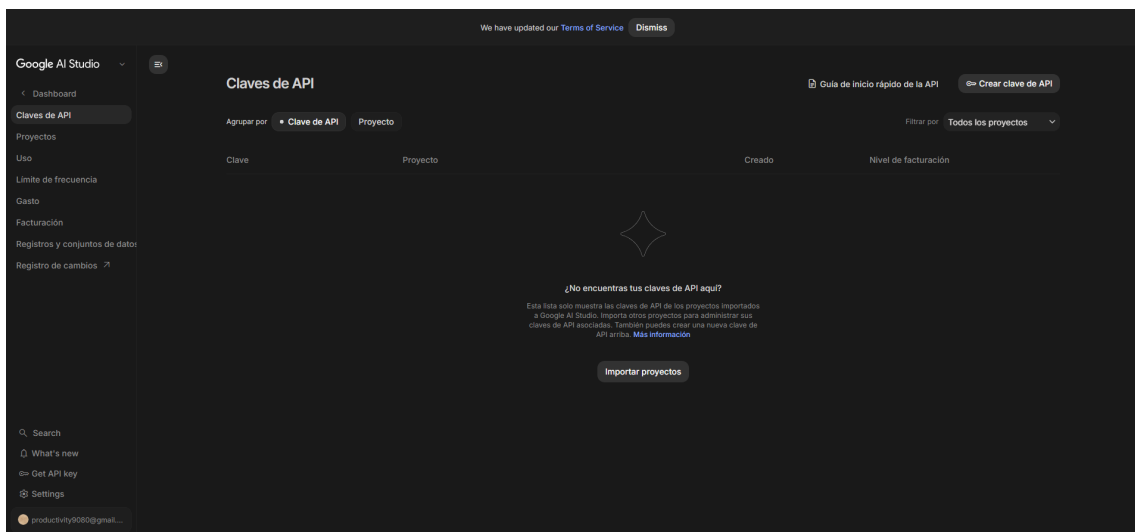
Captura 1.2 — Pantalla principal de Supabase tras iniciar sesión:



Captura 1.3 — Pantalla principal de Google Cloud Console:



Captura 1.4 — Pantalla principal de Google AI Studio:



1.3. Clonado del repositorio

Abra una terminal en el directorio donde desea instalar el proyecto y ejecute:

```
git clone https://github.com/AdriG8/PathFinderAI.git
cd PathFinderAI
```

Captura 1.5 — Terminal mostrando el resultado del `git clone` y el cambio al directorio del proyecto:

```
adrian.gomez@LP-Adrian MINGW64 ~/desktop/test
$ git clone https://github.com/AdriG8/PathFinderAI.git
Cloning into 'PathFinderAI'...
remote: Enumerating objects: 2057, done.
remote: Counting objects: 100% (2057/2057), done.
remote: Compressing objects: 100% (1485/1485), done.
remote: Total 2057 (delta 577), reused 1946 (delta 472), pack-reused 0 (from 0)
Receiving objects: 100% (2057/2057), 7.13 MiB | 4.64 MiB/s, done.
Resolving deltas: 100% (577/577), done.
Updating files: 100% (1577/1577), done.

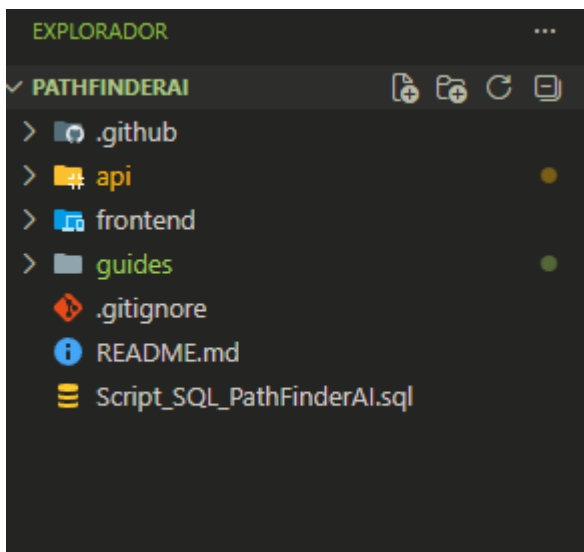
adrian.gomez@LP-Adrian MINGW64 ~/desktop/test
$ ls
PathFinderAI/

adrian.gomez@LP-Adrian MINGW64 ~/desktop/test
$ cd PathFinderAI/

adrian.gomez@LP-Adrian MINGW64 ~/desktop/test/PathFinderAI (main)
$ ls
README.md  Script_SQL_PathFinderAI.sql  api/  frontend/  guides/

adrian.gomez@LP-Adrian MINGW64 ~/desktop/test/PathFinderAI (main)
$
```

Captura 1.6 — Estructura del proyecto abierta en Visual Studio Code (carpetas `api/` y `frontend/`):



2. Configuración del entorno

PathFinderAI se compone de **dos subproyectos**: una API en Node.js (`api/`) y una aplicación React (`frontend/`). Cada uno tiene su propio archivo `.env` con credenciales.

2.1. Crear un proyecto en Supabase

1. Inicie sesión en <https://supabase.com> y pulse **New project**.
2. Asigne un nombre (p. ej. `pathfinderaí`), una contraseña para la base de datos y una región cercana.
3. Espere a que termine el aprovisionamiento (1–2 minutos).
4. En **Project Settings** → **API**, copie:
 - **Project URL** → corresponde a `VITE_SUPABASE_URL` .
 - **anon public** → corresponde a `VITE_SUPABASE_ANON_KEY` .
 - **service_role** → corresponde a `SUPABASE_SERVICE_ROLE_KEY` (⚠ ¡secreta!).

Captura 2.1 — Formulario de creación de un nuevo proyecto en Supabase:

Create a new project

Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization

AdriG8's Org FREE

GitHub (optional)

 Connect GitHub

Ideal for agent-first workflows: update your schema in code, push it to GitHub, and Supabase deploys the changes automatically.
[Learn more](#)

Project name

test

Database password

.....

 Copy

This password is strong. [Generate a password.](#)

Region

 Europe

Select the region closest to your users for the best performance.

Security

☒ **Enable Data API**
Autogenerate a RESTful API for your public schema.
Recommended if using a client library like [supabase-js](#).

☒ **Automatically expose new tables**
Grants privileges to Data API roles by default, exposing new tables.
We recommend disabling this to control access manually.

☐ **Enable automatic RLS**
Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

ADVANCED CONFIGURATION >

Cancel

Create new project

Captura 2.2 — Sección **Project Settings** → **API** con la **Project URL** y la **publishable key**:

Connect to your project

Choose how you want to use Supabase

**Framework**
Use a client library

**Direct**
Connection string

**ORM**
Third-party library

**MCP**
Connect your agent

Framework

React

Variant

Vite

Shadcn

☐

Install components via the Supabase shadcn registry.

Connect your app

Give your agent everything it needs

 Copy prompt

1

Install package
Run this command to install the required dependencies.

```
npm install @supabase/supabase-js
```

2

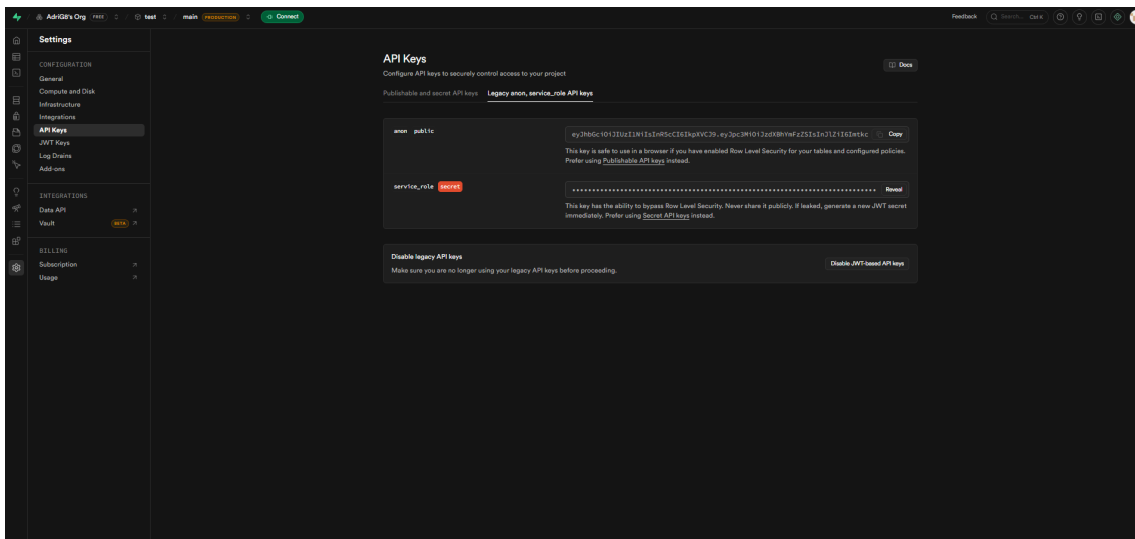
Add files
Add env variables, create a Supabase client, and use it in your app to query data.

```
utils/supabase.ts
```

```
App.tsx
```

```
VITE_SUPABASE_URL=https://kdqprvmguigcfurfsz.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_mzKDXPl9-kthi0M9K-XGvQ_muwdgdz
```

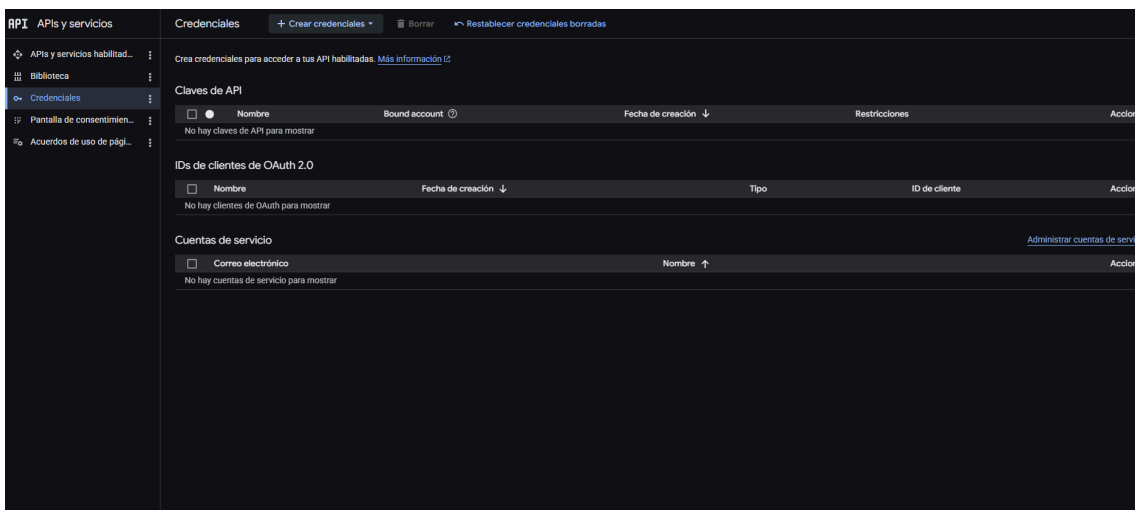
Captura 2.2 (bis) — Claves **anon** y **service_role** del mismo apartado:



2.2. Configurar Google OAuth

1. Acceda a <https://console.cloud.google.com/> y cree o seleccione un proyecto.
2. Vaya a **APIs y servicios** → **Credenciales** y pulse **Crear credenciales** → **ID de cliente OAuth 2.0**.
3. Tipo de aplicación: **Aplicación web**.
4. En **Orígenes JavaScript autorizados** añada:
 - `http://localhost:5173`
5. En **URLs de redirección autorizadas** añada la que Supabase le indique en *Authentication* → *Providers* → *Google* (formato: `https://<su-proyecto>.supabase.co/auth/v1/callback`).
6. Copie el **Client ID** y el **Client Secret**.
7. En el dashboard de Supabase, abra **Authentication** → **Providers** → **Google**, active el proveedor y pegue el Client ID y el Client Secret.

Captura 2.3 — Sección **Credenciales** en Google Cloud:



Captura 2.3 (bis) — Formulario **Crear ID de cliente OAuth 2.0**:

Descripción general | Información de la marca | Público | **Clients** | Acceso a los datos | Centro de verificación | Configuración

Crear ID de cliente de OAuth

Un ID de cliente se usa con el fin de identificar una sola app para los servidores de OAuth de Google. Si la app se ejecuta en varias plataformas, cada una necesitará su propio ID de cliente. Consulta [Configurar OAuth 2.0](#) para obtener más información. [Obtén más información](#) sobre los tipos de clientes de OAuth.

Tipo de aplicación *
Aplicación web

Nombre *
test

El nombre de tu cliente de OAuth 2.0. Este nombre solo se usa para identificar al cliente en la consola y no se muestra a los usuarios finales.

Los dominios de los URI que agregues a continuación se incorporarán automáticamente a la [política de consentimiento de OAuth](#) como [dominios autorizados](#).

Orígenes autorizados de JavaScript

Para usar con solicitudes de un navegador

URI 1 *

+ Agregar URI

URIs de redireccionamiento autorizados

Para usar con solicitudes de un servidor web

URI 1 *

URI 2 *

+ Agregar URI

Nota: La configuración puede tardar entre 5 minutos y algunas horas en aplicarse

Crear Cancelar

Captura 2.4 — URI de redirección autorizado (callback de Supabase) configurado:

Google

Enable Sign-in with Google

Client ID

Client Secret (for OAuth)

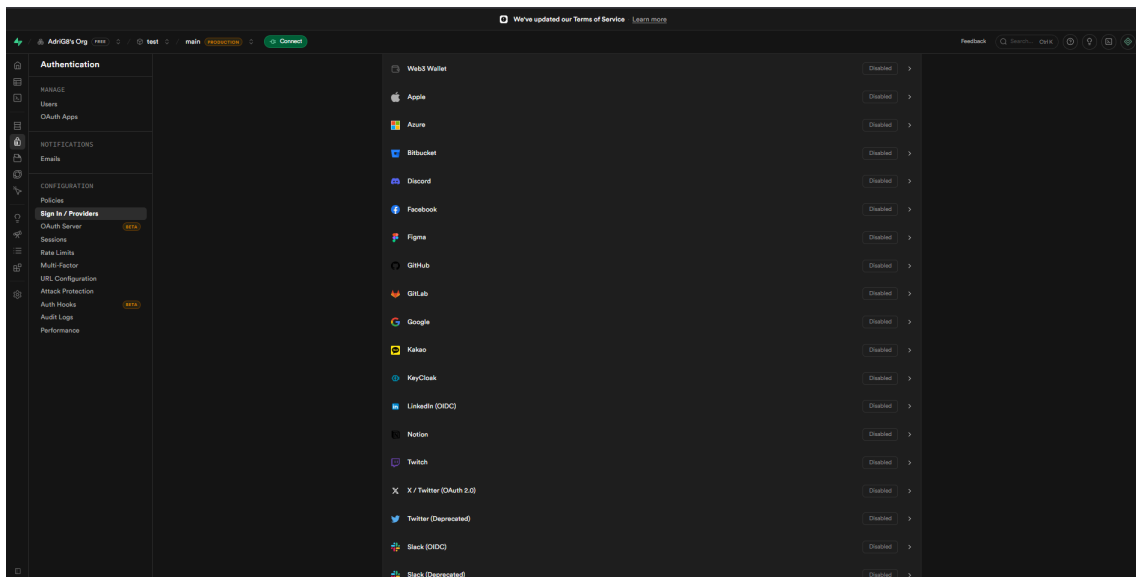
Skip nonce checks

Allow users without an email

Callback URL (for OAuth)

https://fdgpprmgaglfufz.supabase.co/auth/v1/callback

Captura 2.5 — Provider **Google** dentro de **Authentication** → **Providers** de Supabase:



Captura 2.5 (bis) — Client ID y Client Secret obtenidos en Google Cloud:

Se creó el cliente de OAuth

Se puede acceder al ID de cliente desde la pestaña Clientes en Google Auth Platform.



El acceso OAuth está restringido a los [usuarios de prueba](#) que aparecen en la [pantalla de consentimiento de OAuth](#)

ID de cliente

[Redacted]



Ya no podrás ver ni descargar el secreto del cliente una vez que cierres este diálogo. Asegúrate de que copiaste o descargaste la información que aparece a continuación, y de que la almacenaste de forma segura.

Secreto del cliente

[Redacted]

Fecha de creación

21 de mayo de 2026, 10:00:17 a.m.
GMT+2

Estado

✔ Habilitada

⬇ Descargar JSON

Aceptar

Captura 2.5 (ter) — Provider Google activado en Supabase con las credenciales pegadas:

Google

Enable Sign in with Google

Enables Sign in with Google on the web using OAuth or One Tap, or in Android apps or Chrome extensions.

Client IDs

Comma-separated list of client IDs for Web, OAuth, Android apps, One Tap, and Chrome extensions.

Client Secret (for OAuth)

.....

Reveal

Client Secret to use with the OAuth flow on the web.

Skip nonce checks

Allows ID tokens with any nonce to be accepted, which is less secure. Useful in situations where you don't have access to the nonce used to issue the ID token, such as with iOS.

Allow users without an email

Allows the user to successfully authenticate when the provider does not return an email address.

Callback URL (for OAuth)

https://kdqpprvmguiigcfurfsz.supabase.co/auth/v1/callback

Copy

Register this callback URL when using Sign-in with Google on the web using OAuth. [Learn more](#)

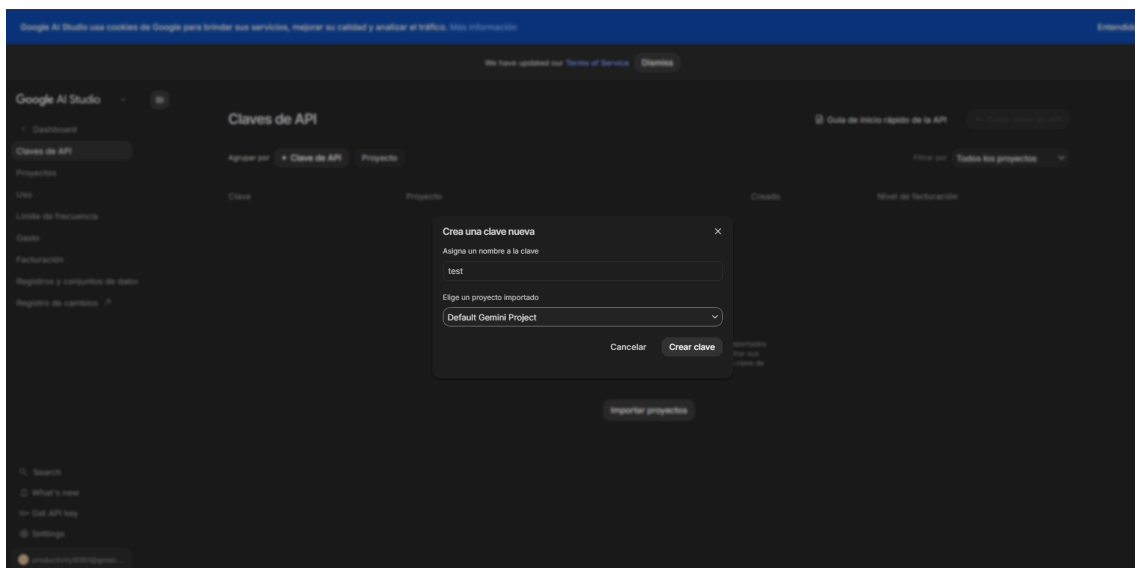
Docs

CancelSave

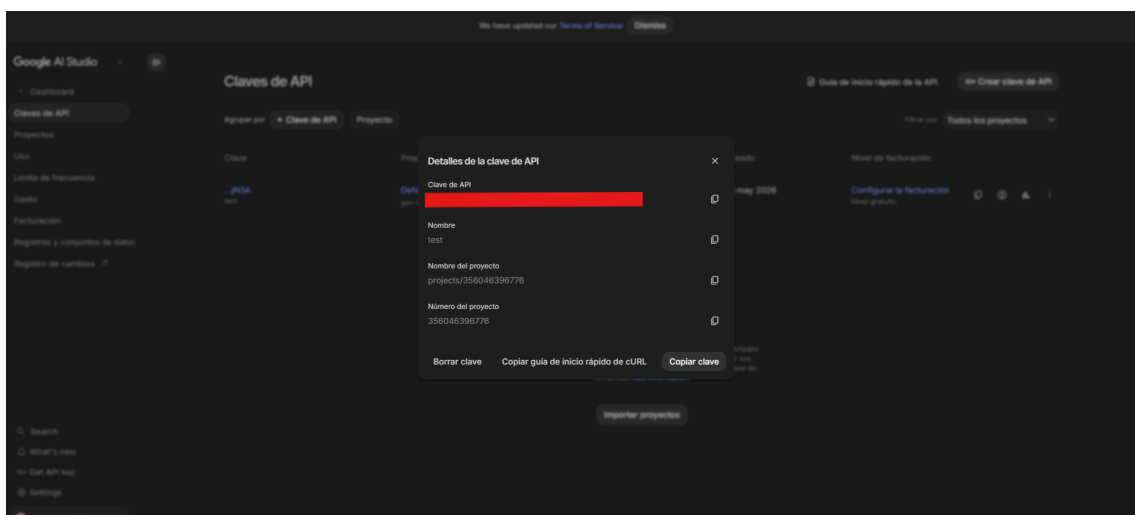
2.3. Obtener la clave de Gemini

1. Acceda a <https://makersuite.google.com/app/apikey>.
2. Pulse **Create API key** y seleccione el proyecto de Google Cloud creado anteriormente.
3. Copie la clave generada — corresponde a `GEMINI_API_KEY` .

Captura 2.6 — Botón **Create API key** en Google AI Studio:



Captura 2.6 (bis) — Clave de API recién generada:



2.4. Configurar la API (`api/.env`)

```
cd api
npm install
```

Cree el archivo `api/.env` con el siguiente contenido y rellene los valores con los obtenidos en los pasos anteriores:

```
# Supabase
VITE_SUPABASE_URL=https://tu-proyecto.supabase.co
VITE_SUPABASE_ANON_KEY=tu-anon-key
SUPABASE_SERVICE_ROLE_KEY=tu-service-role-key

# Google Gemini
```

```
GEMINI_API_KEY=tu-gemini-api-key
```

```
# URL del frontend
```

```
SITE_URL=http://localhost:5173
```

Captura 2.7 — Terminal mostrando el `npm install` completado dentro de `api/` :

```
PS C:\Users\adrian.gomez\Desktop\test\PathFinderAI> cd api
PS C:\Users\adrian.gomez\Desktop\test\PathFinderAI\api> npm i
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead

added 38 packages, and audited 134 packages in 3s

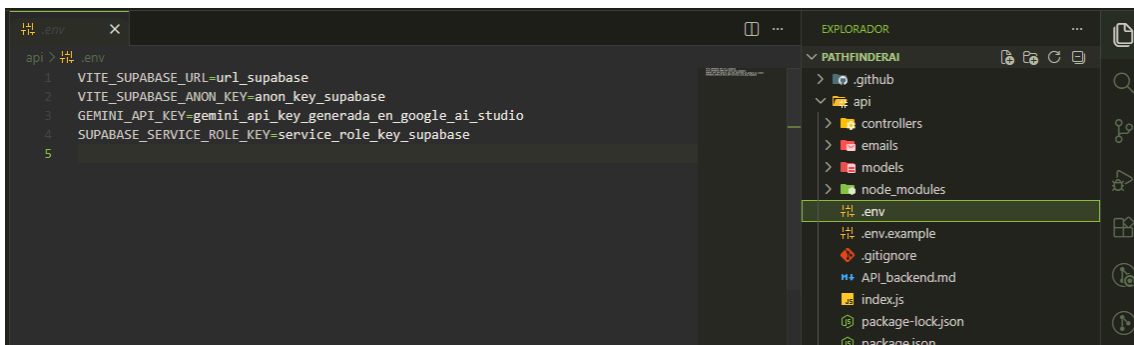
21 packages are looking for funding
  run `npm fund` for details

2 moderate severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
❖ PS C:\Users\adrian.gomez\Desktop\test\PathFinderAI\api>
```

Captura 2.8 — Archivo `api/.env` abierto en VS Code con las variables rellenas:



2.5. Configurar el frontend (`frontend/.env`)

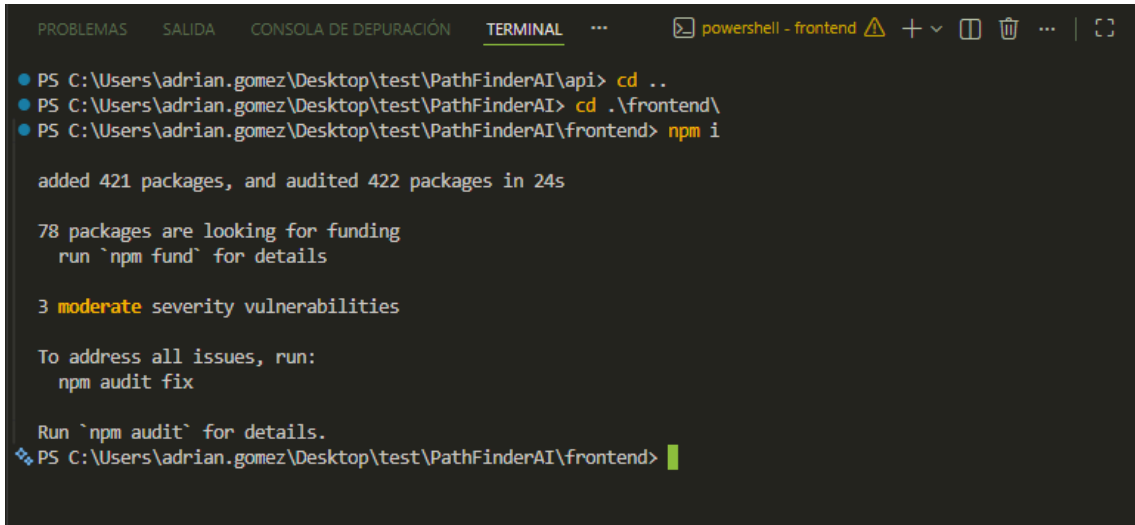
En otra terminal:

```
cd frontend
npm install
```

Cree `frontend/.env` con el siguiente contenido:

```
VITE_API_URL=http://localhost:3000
VITE_SUPABASE_URL=https://tu-proyecto.supabase.co
VITE_SUPABASE_ANON_KEY=tu-anon-key
VITE_GOOGLE_CLIENT_ID=tu-google-client-id.apps.googleusercontent.com
```

Captura 2.9 — Terminal mostrando el `npm install` completado dentro de `frontend/` :



```
PS C:\Users\adrian.gomez\Desktop\test\PathFinderAI\api> cd ..
PS C:\Users\adrian.gomez\Desktop\test\PathFinderAI> cd .\frontend\
PS C:\Users\adrian.gomez\Desktop\test\PathFinderAI\frontend> npm i

added 421 packages, and audited 422 packages in 24s

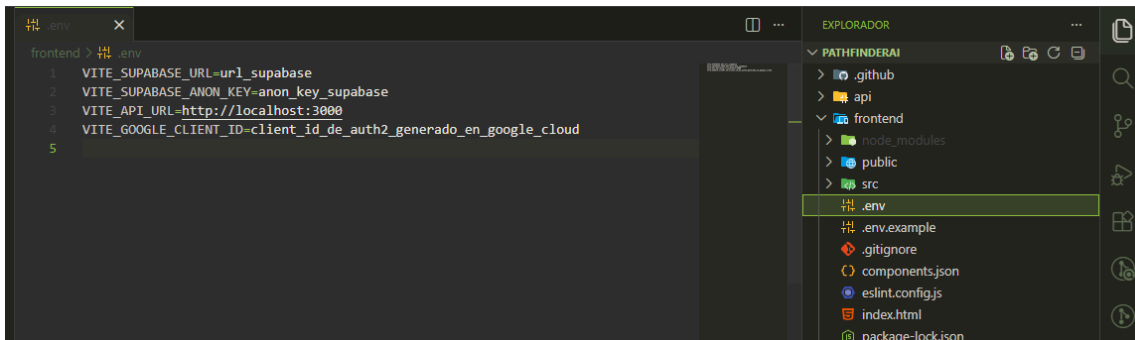
78 packages are looking for funding
  run `npm fund` for details

3 moderate severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\adrian.gomez\Desktop\test\PathFinderAI\frontend>
```

Captura 2.10 — Archivo `frontend/.env` abierto en VS Code con las variables rellenas:



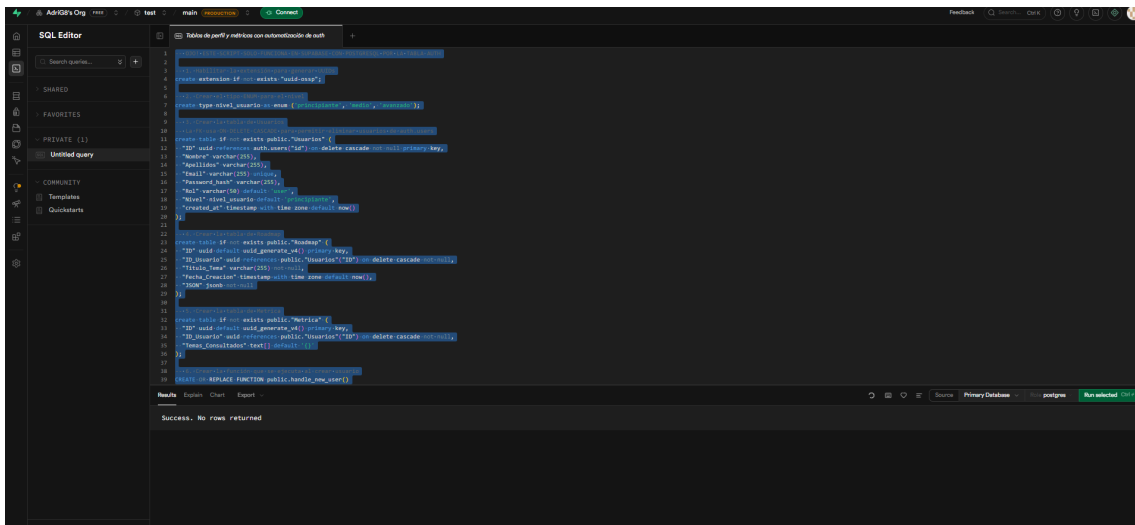
3. Inicialización de la base de datos

PathFinderAI utiliza tres tablas en Supabase: `Usuarios` , `Roadmap` y `Metrica` . El script SQL incluido en el repositorio las crea con todas las restricciones y políticas necesarias.

3.1. Abrir el SQL Editor de Supabase

1. En el dashboard de su proyecto Supabase, pulse el icono **SQL Editor** (lateral izquierdo).
2. Pulse **+ New query** para abrir una pestaña de consulta vacía.

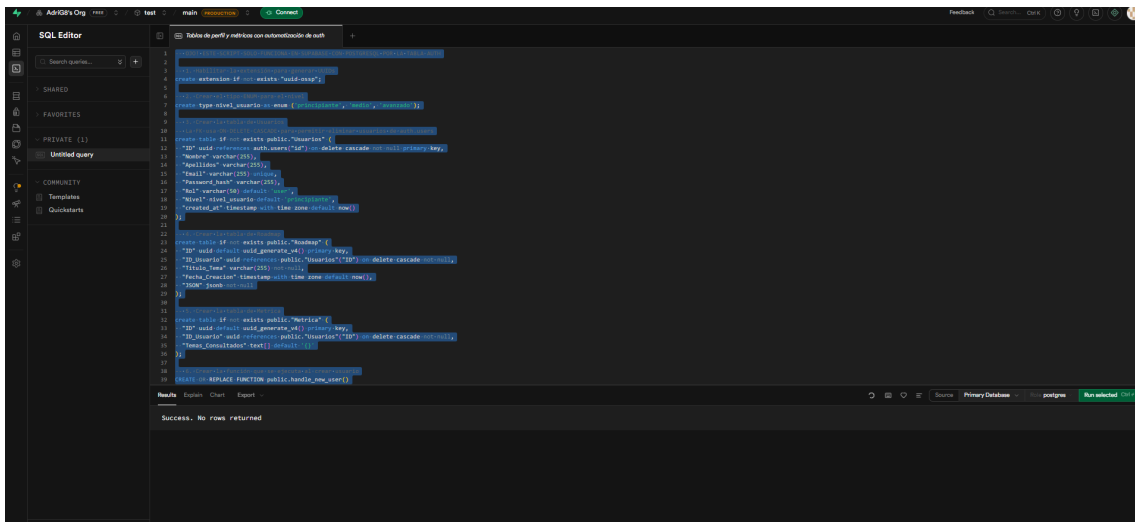
Captura 3.1 — SQL Editor de Supabase:



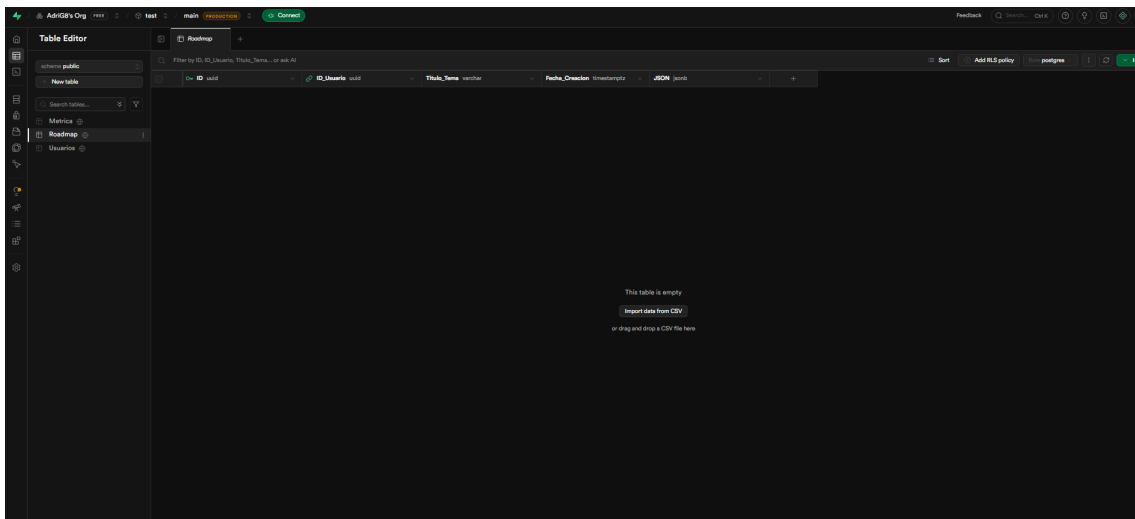
3.2. Ejecutar el script Script_SQL_PathFinderAI.sql

1. Abra el archivo `Script_SQL_PathFinderAI.sql` ubicado en la raíz del repositorio.
2. Copie **todo** su contenido y péguelo en el SQL Editor.
3. Pulse **Run** (o **Ctrl + Enter**).
4. Verifique que en la sección **Results** aparece *Success. No rows returned*.

Captura 3.2 — Script SQL pegado en el editor y ejecutándose:



Captura 3.3 — Tabla `Usuarios` creada correctamente tras ejecutar el script:

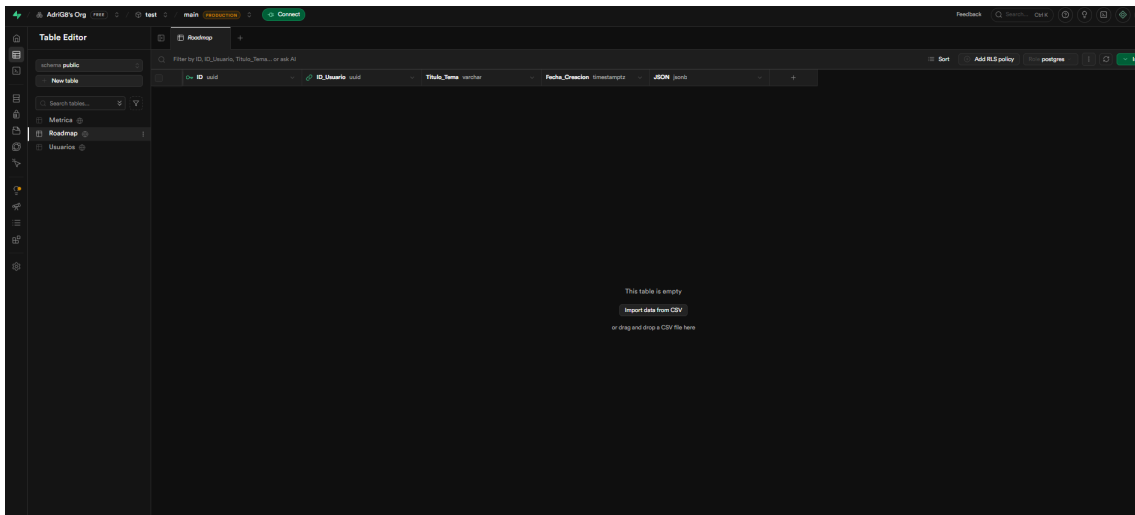


3.3. Comprobar las tablas creadas

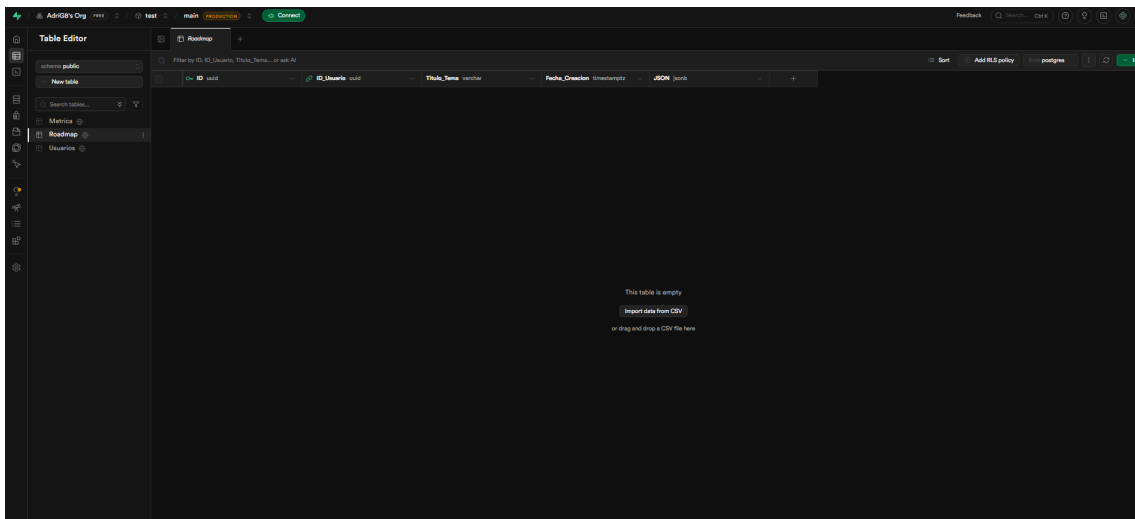
Vaya a **Table Editor** en la barra lateral y confirme que aparecen las tablas:

- Usuarios
- Roadmap
- Metrica

Captura 3.4 — Listado de tablas en el Table Editor de Supabase:



Captura 3.5 — Estructura de la tabla `Roadmap` con sus columnas:



4. Ejecución del sistema

Con las dependencias instaladas y la base de datos lista, arranque los dos servidores **en terminales separadas**.

4.1. Arrancar la API

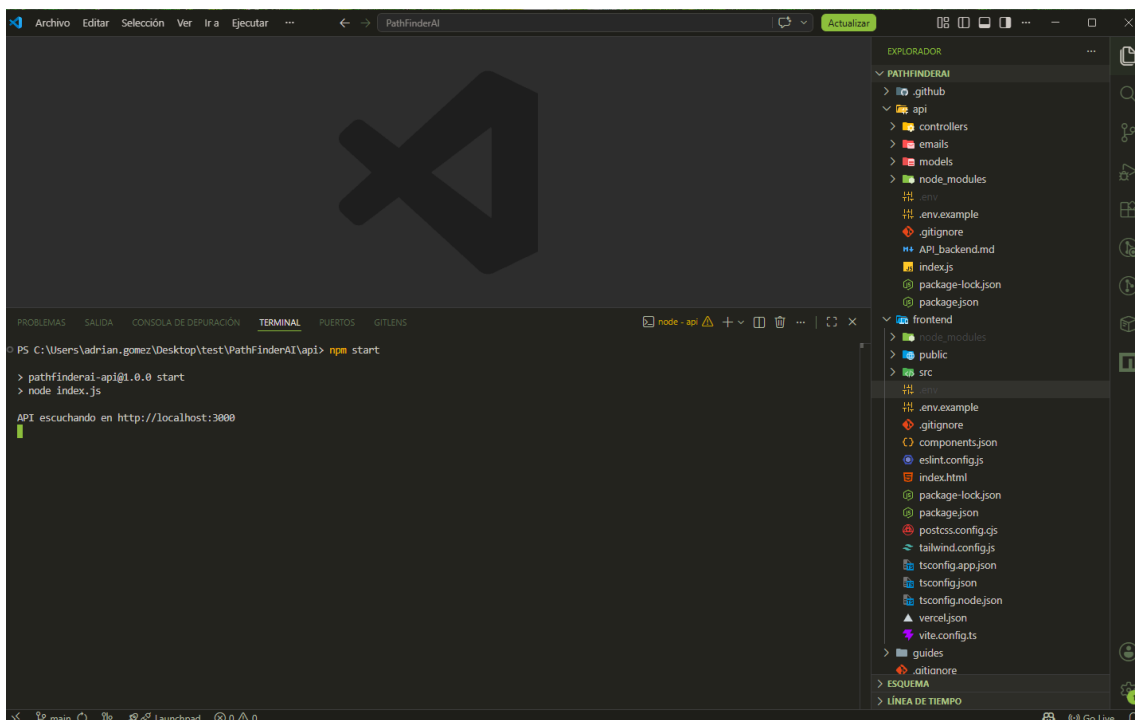
En una terminal posicionada en `api/` :

```
npm run dev
```

La API quedará disponible en <http://localhost:3000>. Debe ver un mensaje como:

🚀 API escuchando en http://localhost:3000

Captura 4.1 — Terminal con la API ejecutándose en el puerto 3000:



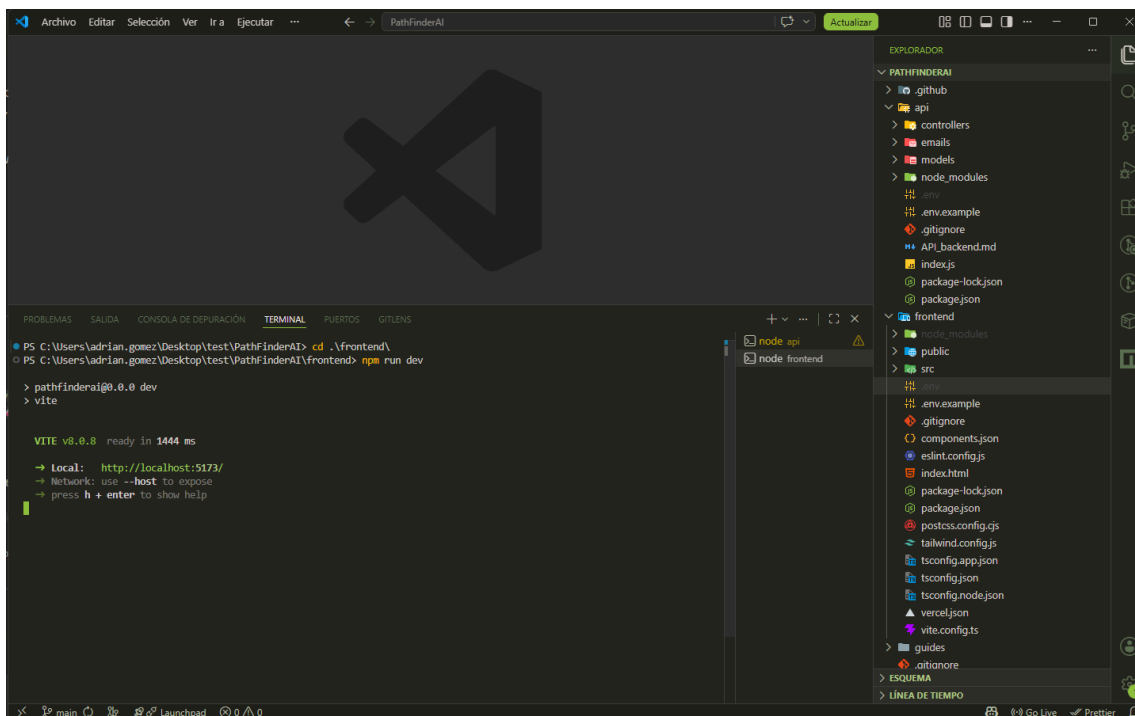
4.2. Arrancar el frontend

En otra terminal posicionada en `frontend/` :

```
npm run dev
```

Vite mostrará la URL local (normalmente <http://localhost:5173>).

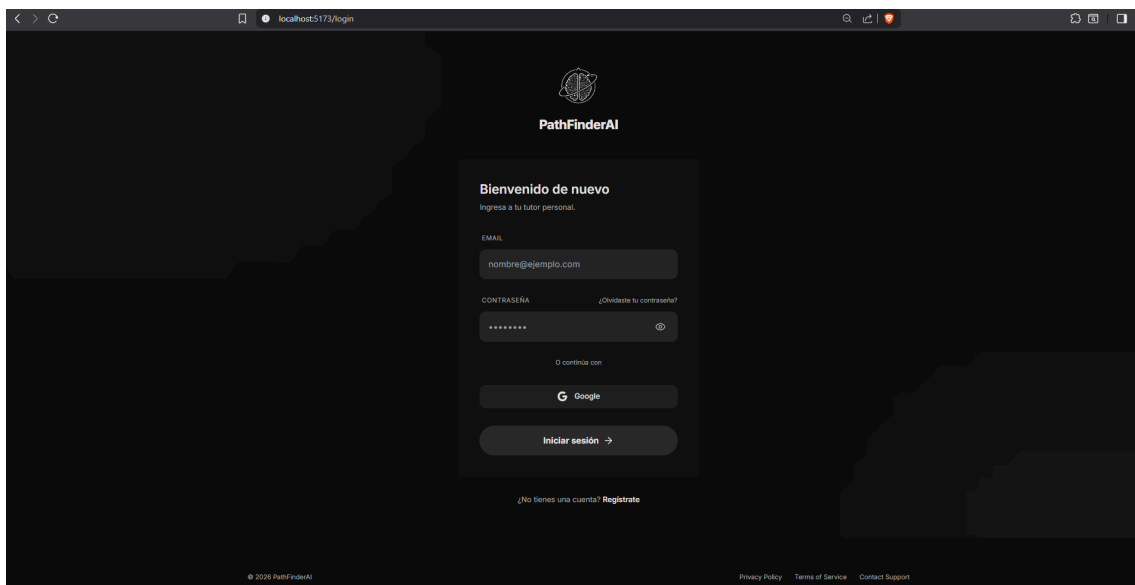
Captura 4.2 — Terminal con Vite ejecutándose y mostrando la URL del frontend:



4.3. Abrir la aplicación en el navegador

Acceda a <http://localhost:5173>. Será redirigido a la pantalla de inicio de sesión.

Captura 4.3 — Pantalla de login de PathFinderAI cargada en el navegador:



5. Ejecución de tests

PathFinderAI incluye **80 tests** (unitarios y de integración) que verifican el correcto funcionamiento de la aplicación.

5.1. Tests del Backend (Jest)

Desde una terminal posicionada en `api/` :

```
npm test
```

Este comando ejecuta las 7 suites de tests (66 tests en total) con Jest:

Suite	Tipo	Tests
generateController.test.js	Unitario	10
examController.test.js	Unitario	9
UserController.test.js	Unitario	13
roadmapController.test.js	Unitario	10
adminController.test.js	Unitario	5
backendDb.test.js	Integración	6
frontendBackend.test.js	Integración	13

5.2. Tests del Frontend (Vitest)

Desde una terminal posicionada en `frontend/` :

```
npx vitest run
```

Este comando ejecuta 1 suite de tests (14 tests en total) con Vitest:

Suite	Tipo	Tests
sanitize.test.ts	Unitario	14

5.3. Resultado esperado

Todos los tests deben pasar correctamente:

```
Backend:  Test Suites: 7 passed – Tests: 66 passed
Frontend: Test Files:  1 passed – Tests: 14 passed
```

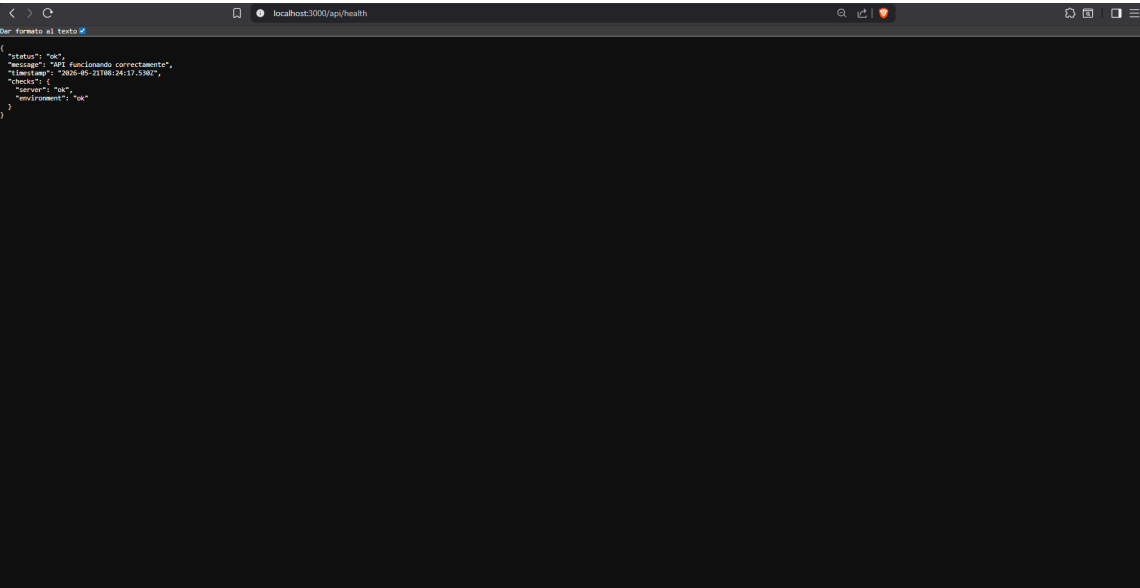
6. Verificación final

Para confirmar que la instalación es correcta, realice esta lista de comprobación:

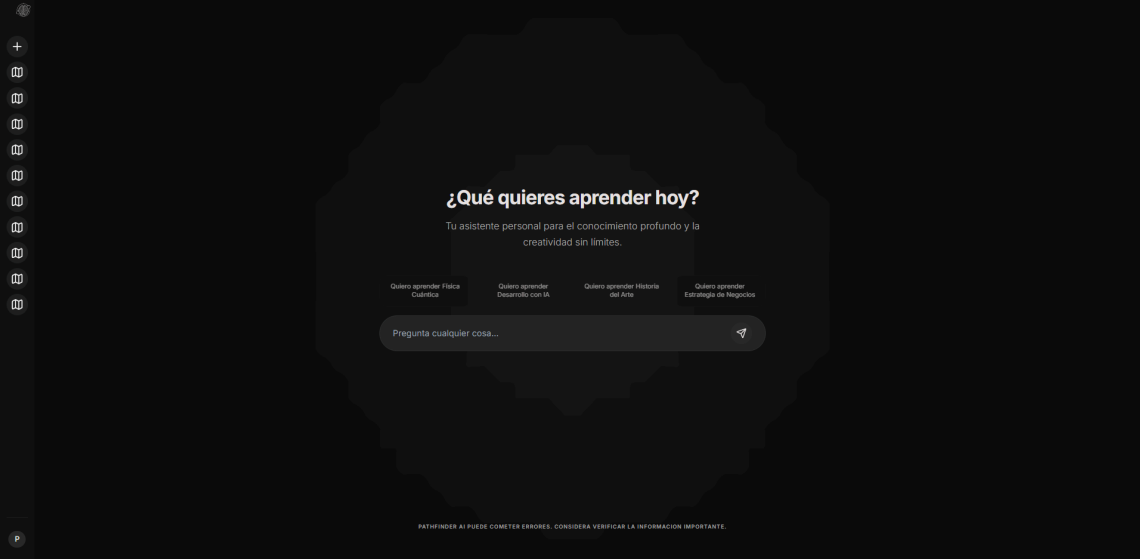
#	Comprobación	Resultado esperado
1	GET <code>http://localhost:3000/api/health</code>	<code>{ "status": "ok" }</code>
2	Página <code>/register</code> accesible	Formulario de registro visible

3	Registrar un usuario nuevo	Se recibe el email de confirmación
4	Iniciar sesión con el usuario creado	Redirección al dashboard
5	Generar un roadmap (p. ej. " <i>electricidad</i> ")	Aparece el grafo con varios nodos
6	Tabla Usuarios en Supabase	Contiene el usuario registrado
7	Tabla Roadmap en Supabase	Contiene el roadmap generado

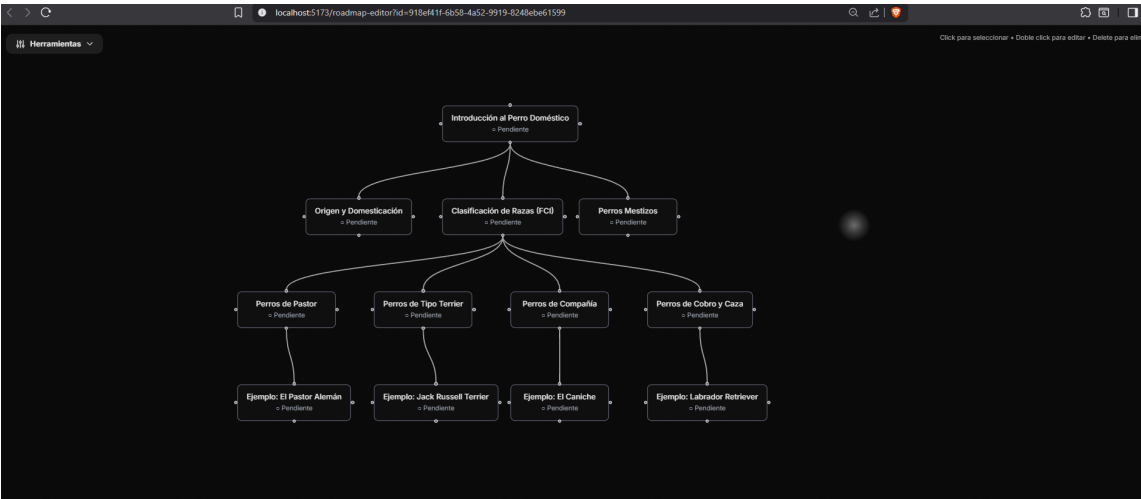
Captura 5.1 — Respuesta JSON del endpoint `/api/health` en el navegador:



Captura 5.2 — Dashboard de PathFinderAI tras iniciar sesión:



Captura 5.3 — Roadmap recién generado y visualizado en el editor:



Captura 5.4 — Registro insertado en la tabla `Roadmap` de Supabase:

 Fila en tabla Roadmap

7. Resolución de problemas

Síntoma	Causa probable	Solución
EADDRINUSE: port 3000 already in use	Hay otro proceso usando el puerto	Cierre el proceso o cambie el puerto en <code>api/index.js</code> .
Invalid API key al generar un roadmap	GEMINI_API_KEY incorrecta o ausente	Revise <code>api/.env</code> y reinicie la API.
Login con Google falla	URI de redirección no autorizado en Google Cloud o Supabase	Añada <code>http://localhost:5173</code> en orígenes y la URL de callback de Supabase.
Error relation "Usuarios" does not exist	El script SQL no se ejecutó completo	Vuelva a ejecutar <code>Script_SQL_PathFinderAI.sql</code> desde el SQL Editor.
CORS bloqueado en el navegador	SITE_URL en <code>api/.env</code> no coincide con el frontend	Ajuste <code>SITE_URL=http://localhost:5173</code> y reinicie la API.
npm install falla con ERESOLVE	Versión de Node antigua	Actualice Node.js a la versión 18 o superior.

✅ ¡Instalación completada!

Continúe con el [Manual de Usuario](#) para aprender a utilizar PathFinderAI.